Assignment 7: Threads, Synchronization, Deadlock Detection and CPU Scheduling

Name: Mehal Srivastava
2401MC57

1. Introduction

This assignment implements user-level threads, semaphore synchronization, deadlock detection and avoidance, and priority-based preemptive CPU scheduling in xv6-riscv.

2. Question 1: User-Level Threads

The `clone()` system call creates a thread that shares its creator's address space but has its own execution context and stack. The `join()` system call waits for a thread to finish and releases its resources. The process structure and cleanup routines (`exit()`, `fork()`, and `wait()`) must distinguish threads from independent processes.

The `threadtest.c` program creates three threads, each incrementing a shared counter 100,000 times. The expected value is 300,000. Without synchronization, race conditions may cause lost increments.

3. Question 2: Semaphore Synchronization

Counting semaphores are implemented using `sem_init()`, `sem_wait()`, and `sem_signal()`. Kernel synchronization primitives such as spinlocks, `sleep()`, and `wakeup()` protect semaphore operations.

In `synctest.c`, semaphore operations protect the shared counter, ensuring the final value should be 300,000. The Producer–Consumer problem uses a buffer of five items and three semaphores:

- EMPTY = 5: Available slots.
- FULL = 0: Occupied slots.
- MUTEX = 1: Protects the buffer.

These semaphores prevent buffer overflow, underflow, and simultaneous unsafe access.

4. Question 3: Deadlock Detection and Avoidance

The `deadlock.c` program uses `Allocation`, `Request`, and `Available` to detect deadlocks. It repeatedly finds processes whose requests can be satisfied, assumes they

finish, and releases their allocated resources. Any unfinished processes remaining may be deadlocked.

The Banker's algorithm checks system safety using:

Need = Max − Allocation

A request is granted only if resources are available and the resulting state remains safe. Testing should cover safe, deadlocked, and unsafe-after-request states.

5. Question 4: Priority-Based Preemptive Scheduling

An integer priority is added to each process. Priority 1 is highest, 20 is lowest, and new processes receive priority 10. The `set_priority(pid, priority)` and `get_priority(pid)` system calls modify and retrieve priorities.

The scheduler selects the RUNNABLE process with the lowest numerical priority, resolving ties using round-robin order. To prevent starvation, aging improves a waiting process's priority by decreasing its numerical value by one after every 30 waiting ticks, with a minimum priority of 1.

The `priotest.c` program tests processes with priorities 1, 5, 10, and 20. Actual execution order and priority changes should be recorded from terminal output.

6. Challenges

Thread implementation requires careful shared-memory and resource management. Semaphore operations must be correctly ordered to avoid race conditions and deadlocks. Deadlock algorithms require accurate resource tracking, while priority scheduling needs aging to prevent starvation.

7. Conclusion

This assignment demonstrates thread management, synchronization using semaphores, deadlock detection and avoidance, and priority-based CPU scheduling with aging in xv6-riscv. These implementations illustrate key operating system concepts involving concurrency, resource allocation, and process scheduling.

8. Submission Evidence

Attach:

1. `threadtest.c` output showing the shared counter.
2. `synctest.c` output for synchronization and Producer–Consumer execution.
3. `deadlock.c` output for all test cases.

4. `priotest.c` output showing priorities, aging, and completion order.
5. Relevant kernel source files and a successful xv6 build log.